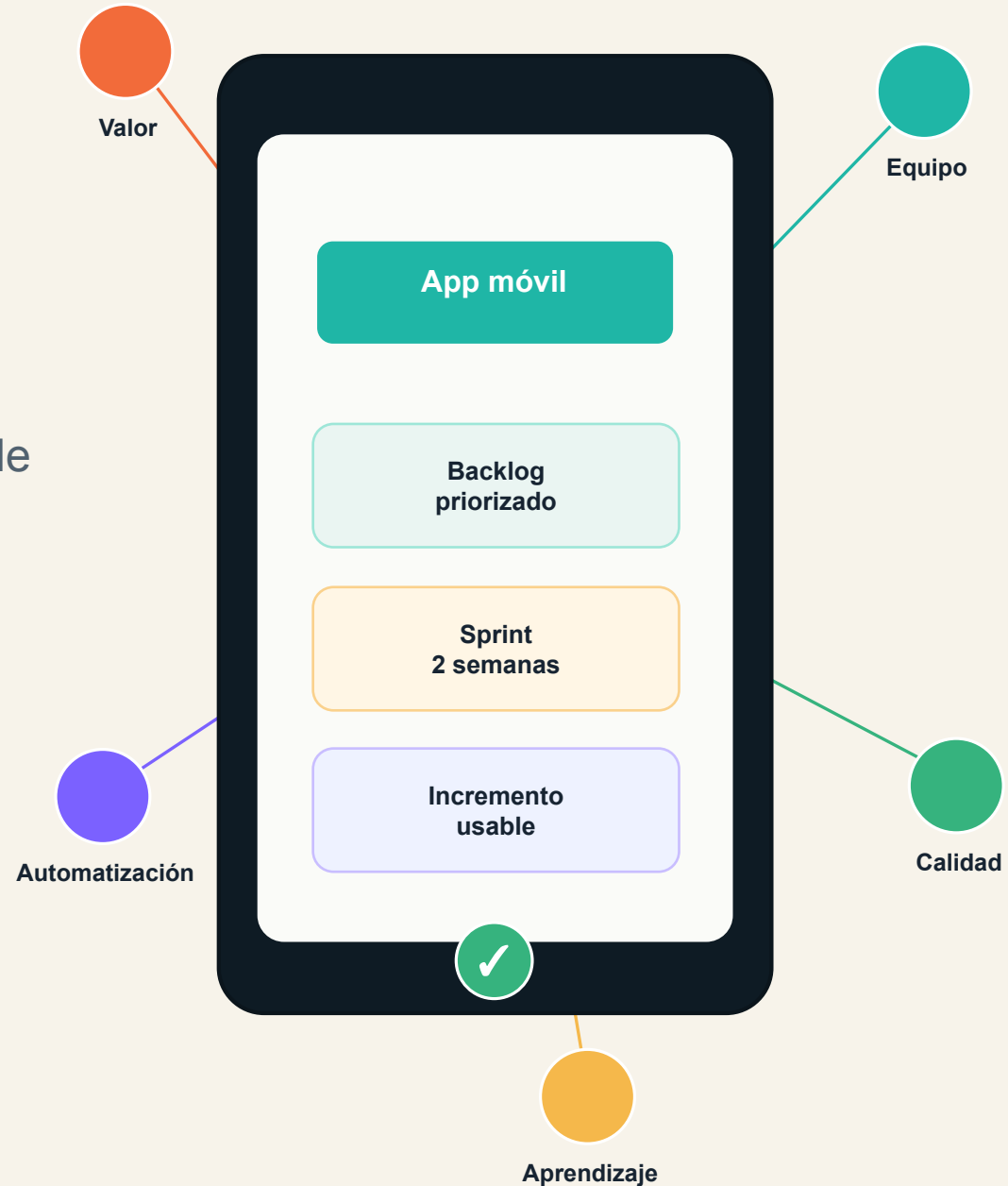


Introducir Scrum en una startup tecnológica

Una propuesta práctica para mejorar entrega de valor, calidad, colaboración y adaptación

Scrum como sistema de trabajo para entregar mejor producto, no como moda metodológica.



Cinco preguntas, cinco decisiones de trabajo

La adopción de Scrum se articula alrededor de cinco necesidades prácticas del equipo.

1

Inicio

2

Adopción

3

Calidad

4

Automatización

5

Colaboración

Pregunta del caso

Decisión propuesta

¿Cómo introducir Scrum?

Propósito, roles, backlog y calidad

¿Cómo gestionar resistencias?

Escuchar, formar y piloto

¿Qué hacer si baja la calidad?

Definition of *Done* y prácticas técnicas

¿Cómo evaluar automatización?

Medir impacto antes de escalar

¿Cómo colaborar en remoto?

Herramientas, acuerdos y eventos

1. Empezar por el propósito, no por las ceremonias

El objetivo no es “hacer Scrum”, sino mejorar valor, colaboración y adaptación.

Primeros pasos recomendados

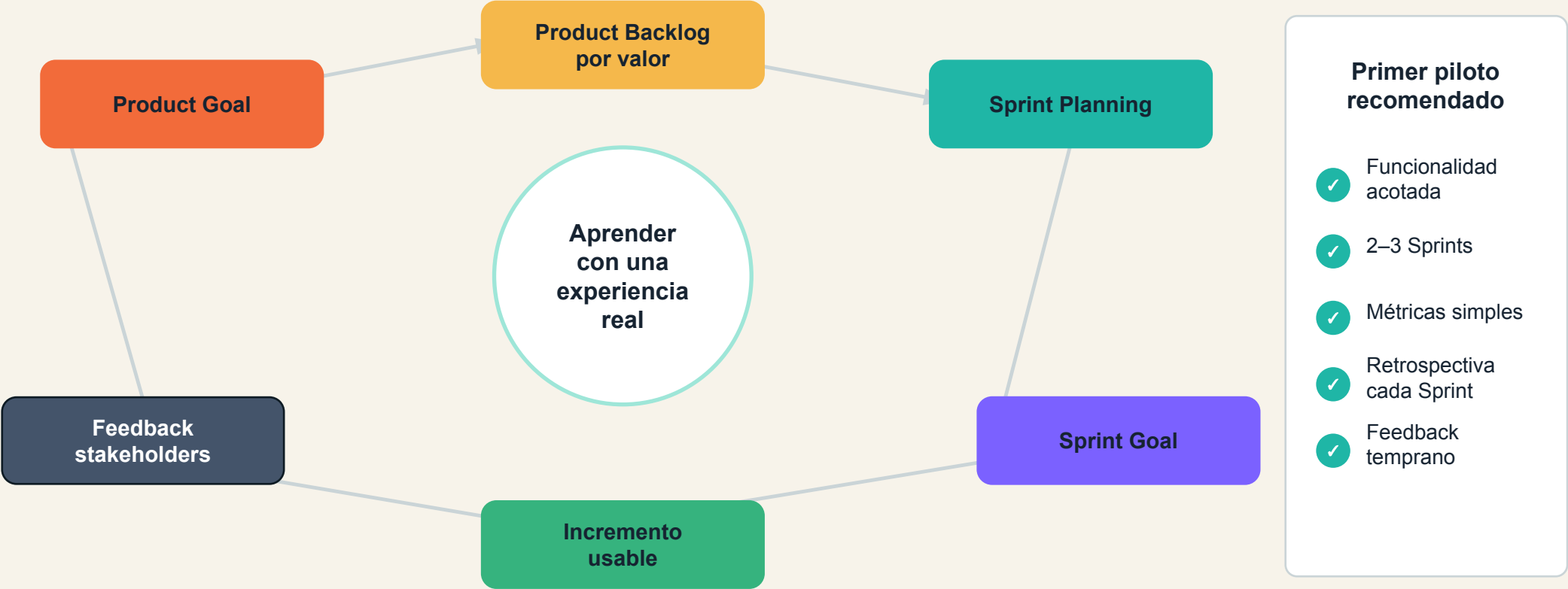
- Clarificar el propósito del cambio.
- Definir un Product Goal inicial para la app móvil.
- Aclarar responsabilidades: Product Owner, Scrum Master y Developers.
- Construir un Product Backlog inicial ordenado por valor.
- Acordar Definition of Done, duración de Sprints y tablero visual sencillo.



Antes de arrancar Sprints, el equipo necesita dirección clara, responsabilidades entendidas y una definición compartida de “hecho”.

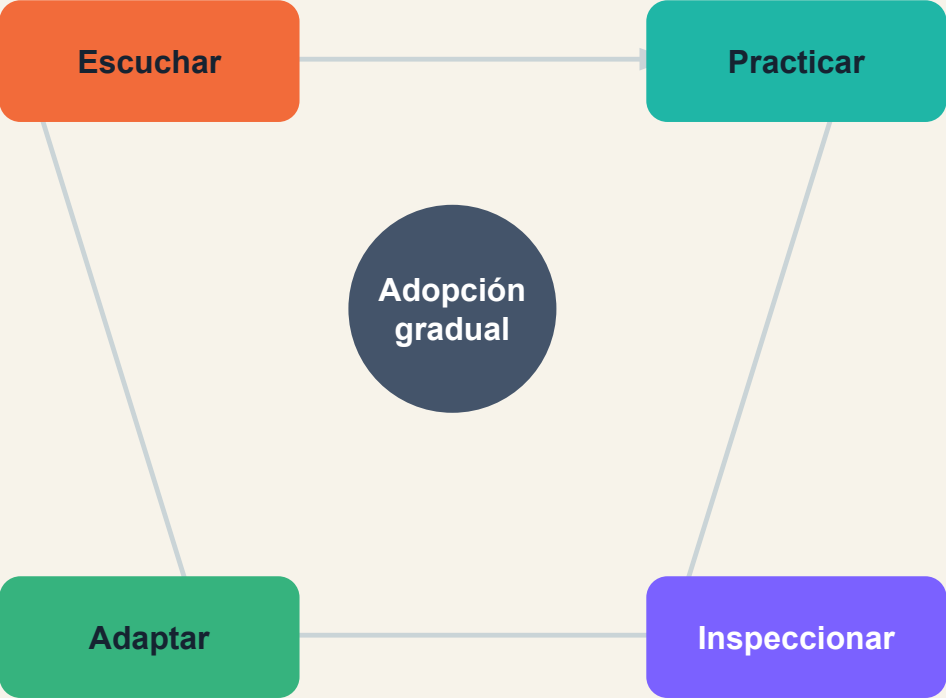
De la intención al incremento usable

Scrum convierte objetivos de producto en entregas pequeñas, inspeccionables y útiles.



2. Gestionar la resistencia como información útil

El cambio funciona mejor cuando el equipo lo experimenta, no cuando se le impone.



Estrategia de adopción

Escucha activa	Entender dudas y expectativas
Formación práctica	Aprender desde casos reales
Piloto de varios Sprints	Practicar en funcionalidad acotada
Retrospectivas	Inspeccionar y adaptar
Hackathones	Laboratorio seguro de aprendizaje

Los hackathones pueden apoyar el aprendizaje, pero no sustituyen la práctica real de Scrum en el producto principal.

3. Completar tareas no equivale a entregar calidad

Si la calidad baja, no estamos avanzando: estamos acumulando deuda.

Señal de alerta:
las tareas se completan, pero el producto no alcanza la calidad esperada.

Respuesta propuesta

- ✓ Revisar Definition of Done
- ✓ Aclarar criterios de aceptación
- ✓ Mejorar prácticas de testing
- ✓ Revisar deuda técnica
- ✓ Analizar presión de alcance
- ✓ Llevarlo a la Retrospective



Mejoras concretas:

- Pruebas automatizadas
- Revisión de código
- Menos trabajo en curso
- Refactorización planificada

La calidad no se sacrifica para aparentar avance.

4. Automatizar donde haya impacto medible

La automatización debe priorizarse por valor, calidad, tiempo ahorrado y reducción de errores.

Criterios de evaluación

- 1 Valor para el producto
- 2 Mejora de calidad
- 3 Tiempo ahorrado
- 4 Reducción de errores
- 5 Coste y complejidad
- 6 Riesgo de implementación



Decisión recomendada: experimento pequeño, medible y revisable antes de comprometer varios Sprints completos.

5. No basta con estar conectados: necesitamos transparencia y confianza

Las herramientas ayudan, pero la colaboración nace de acuerdos, eventos útiles y aprendizaje continuo.

Herramientas digitales

Mensajería · tablero visual · documentación ligera

Eventos Scrum útiles

Daily Scrum · Refinement · Review · Retrospective

Acuerdos de trabajo remoto

Canales claros · tiempos de respuesta · normas síncronas/asíncronas

Confianza

Transparencia · feedback · aprendizaje continuo

La colaboración remota no se resuelve solo con Slack, Trello o Jira. Se diseña mediante acuerdos claros, cadencias compartidas y espacios reales de inspección y adaptación.

No queremos que el equipo simplemente “haga Scrum”; queremos que Scrum ayude al equipo a crear mejor producto, con más foco, más calidad y más capacidad de adaptación.

